

Práctica 3 del Tema 4

Víctor Manuel Peiró Martínez

FIIS 2ºA

Programación Orientada a Objetos

Clase CuentaPersonal

```
package tema4.practica3;
public class CuentaPersonal {

    // Atributos y Constantes
    public static final Double DEFAULT_SALDO = 0.0;

    protected Integer numeroCuenta;
    protected String nombreCliente;
    protected Double saldo;

    //Constructores

    //Constructor con 2 parametros
    public CuentaPersonal(Integer numeroCuenta, String nombreCliente) {
        this(numeroCuenta, nombreCliente, CuentaPersonal.DEFAULT_SALDO);
    }

    //Constructor Principal
    public CuentaPersonal(Integer numeroCuenta, String nombreCliente, Double saldo) {
        this.numeroCuenta = numeroCuenta;
        this.nombreCliente = nombreCliente;
        this.saldo = saldo;
    }

    //Getters y Setters
    public Integer getNumeroCuenta() {
        return this.numeroCuenta;
    }
    public void setNumeroCuenta(Integer numeroCuenta) {
        this.numeroCuenta = numeroCuenta;
    }
    public String getNombreCliente() {
        return this.nombreCliente;
    }
    public void setNombreCliente(String nombreCliente) {
        this.nombreCliente = nombreCliente;
    }
    public Double getSaldo() {
        return this.saldo;
    }
    public void setSaldo(Double saldo) {
        this.saldo = saldo;
    }

    //To string
    public String toString() {
        return "CuentaPersonal [numeroCuenta=" + this.numeroCuenta + ", nombreCliente="
+ this.nombreCliente + ", saldo=" + this.saldo+ "]\n";
    }
}
```

```

//Imprime el toString
public void consultarDatos() {
    System.out.println(this.toString());
}

//Añade un cantidad al saldo
public void depositar(Double cantidad) {
    this.saldo += cantidad;
}

//Retirar cantidad de tu saldo
public void retirar(Double cantidad) {
    //Solo puedes retirar si posees esa cantidad de dinero
    if (this.saldo >= cantidad) {
        this.saldo -= cantidad;
    } else {
        System.out.println("ERROR: No tienes tanto dinero");
    }
}

//Compara 2 cuentas personales
public boolean equals(Object object) {
    boolean iguales = false;
    //Veo si mi objeto a comparar es una cuenta personal
    if (object instanceof CuentaPersonal) {
        CuentaPersonal cuenta1 = (CuentaPersonal)object; //Hago downcasting
        iguales = this.numeroCuenta.equals(cuenta1.numeroCuenta); //Veo si son
        iguales
    }

    return iguales;
}
}

```

Clase CuentaAhorros

```
package tema4.practica3;
import tema3.Fecha;
//Hereda de CuentaPersonal
public class CuentaAhorros extends CuentaPersonal{
    //Atributos
    private Fecha fechaVencimiento;
    private Double interesMensual;

    //Constructor
    public CuentaAhorros(Fecha fechaVencimiento, Double interesMensual ,Integer
numeroCuenta, String nombreCliente, Double saldo) {
        super(numeroCuenta, nombreCliente,saldo);
        this.fechaVencimiento = fechaVencimiento;
        this.interesMensual = interesMensual;
    }

    //Getters y Setters
    public Fecha getFechaVencimiento() {
        return fechaVencimiento;
    }
    public void setFechaVencimiento(Fecha fechaVencimiento) {
        this.fechaVencimiento = fechaVencimiento;
    }
    public Double getInteresMensual() {
        return interesMensual;
    }
    public void setInteresMensual(Double interesMensual) {
        this.interesMensual = interesMensual;
    }

    //Metodo toString
    public String toString() {
        return super.toString() + "[" + this.fechaVencimiento + ", " + this.interesMensual + "]";
    }

    //Inserta intereses solo si es el primer dia del mes
    public void insertarIntereses() {
        if (new Fecha().getDia().equals(1)) {
            super.depositar(super.saldo * this.interesMensual);
        }
    }

    //Sobrescritura del metodo retirar solo para que se pueda retirar el ultimo dia del mes
    public void retirar(Double cantidad, Fecha fechaOperacion) {
        if(fechaOperacion.equals(fechaVencimiento)) {
            super.retirar(cantidad);
        } else {
            System.out.println("ERROR: Solo se puede retirar dinero el dia de la fecha
de vencimiento");
        }
    }
}
```

Clase CuentaCheque

```
package tema4.practica3;
import tema3.Fecha;
public class CuentaCheque extends CuentaPersonal{

    //Atributos y Constantes
    public static final Double DEFAULT_COMISION_CHEQUERA = 0.5;
    public static final Double DEFAULT_COMISION_SALDO_INDEPENDIENTE = 0.05;

    private Boolean numerosRojos;
    private Double comisionChequera;
    private Double comisionSaldoInsuficiente;

    //Constructores
    //Constructor de 2 parametros
    public CuentaCheque(Integer numeroCuenta, String nombreCliente) {
        this(numeroCuenta,nombreCliente, CuentaPersonal.DEFAULT_SALDO,
CuentaCheque.DEFAULT_COMISION_CHEQUERA,
CuentaCheque.DEFAULT_COMISION_SALDO_INDEPENDIENTE);
    }

    //Constructor Principal
    public CuentaCheque(Integer numeroCuenta,String nombreCliente ,Double saldo,Double
comisionChequera, Double comisionSaldoInsuficiente) {
        super(numeroCuenta,nombreCliente,saldo); //Constructor para la clase padre
        this.comisionChequera = comisionChequera;
        this.comisionSaldoInsuficiente = comisionSaldoInsuficiente;
        this.numerosRojos = super.saldo < 0; //El valor de numeros rojos depende del estado
del saldo
    }

    //Getters y setters
    public Boolean getNumerosRojos() {
        return numerosRojos;
    }
    public void setNumerosRojos(Boolean numerosRojos) {
        this.numerosRojos = numerosRojos;
    }
    public Double getComisionChequera() {
        return comisionChequera;
    }
    public void setComisionChequera(Double comisionChequera) {
        this.comisionChequera = comisionChequera;
    }
    public Double getComisionSaldoInsuficiente() {
        return comisionSaldoInsuficiente;
    }
    public void setComisionSaldoInsuficiente(Double comisionSaldoInsuficiente) {
        this.comisionSaldoInsuficiente = comisionSaldoInsuficiente;
    }

    //Metodo toString
```

```

    public String toString() {
        return super.toString() + "CuentaCheque [numerosRojos=" + numerosRojos + ",
comisionChequera=" + comisionChequera
            + ", comisionSaldoInsuficiente=" + comisionSaldoInsuficiente + "];
    }
    //Resta dinero al saldo por tener una comision chequera
    public void comisionChequera() {
        if (new Fecha().getDia().equals(1)) {
            super.saldo -= super.saldo*this.comisionChequera;
        }
    }

    //Sobreescribe el metodo retirar para que pueda retirar sin importar que puede tener numeros
    rojos
    @Override
    public void retirar(Double cantidad) {
        super.saldo -= cantidad;
        if (super.saldo < cantidad) {
            super.saldo -= super.saldo * this.comisionSaldoInsuficiente;
            this.numerosRojos = true;
        }
        super.saldo -= cantidad;
    }

    //Sobreescribe el metodo depositar para que actualice el atributo de numeros rojos
    @Override
    public void depositar(Double cantidad) {
        super.depositar(cantidad);
        this.numerosRojos = super.saldo < 0;
    }
}

```

Clase RegistroCuentas

```
package tema4.practica3;

import java.util.ArrayList;
import java.util.List;

public class RegistroCuentas {

    //Atributo
    private List<CuentaPersonal> registro;

    //Constructor
    public RegistroCuentas() {
        this.registro = new ArrayList<CuentaPersonal>();
    }

    //Añade una cuenta a su lista
    public Boolean insertarCuenta(CuentaPersonal cuenta) {
        return this.registro.add(cuenta);
    }

    //Elimina una cuenta de su lista y devuelve si ha conseguido eliminarla
    public Boolean borrarCuenta(CuentaPersonal cuenta) {
        Boolean encontrado = false;
        for (CuentaPersonal acc: this.registro) {
            if (acc.equals(cuenta)) {
                this.registro.remove(cuenta);
                encontrado = true;
            }
        }
        return encontrado;
    }

    //Busca una cuenta y la devuelve si la encuentra, sino devuelve null
    public CuentaPersonal buscarCuenta (Integer numCuenta) {

        CuentaPersonal cuenta = null;

        for (CuentaPersonal acc : this.registro) {
            if (acc.getNumeroCuenta().equals(numCuenta)) {
                cuenta = acc;
            }
        }
        return cuenta;
    }
}
```

Clase PruebaCuentas

```
package tema4.practica3;
import tema3.Fecha;
public class PruebaCuentas {
    public static void main(String[] args) {
        CuentaPersonal cuentaDePaco = new CuentaPersonal(1, "Paco", 10.0);
        CuentaPersonal cuentaDeJose = new CuentaAhorros(new Fecha(31,12,2022),2.0,2,
"Jose", 20.0);
        CuentaPersonal cuentaDePilar = new CuentaCheque(3, "Pilar", 30.0, 3.0, 0.05);
        RegistroCuentas registroDeCuentas = new RegistroCuentas();
        registroDeCuentas.insertarCuenta(cuentaDePaco);
        registroDeCuentas.insertarCuenta(cuentaDeJose);
        registroDeCuentas.insertarCuenta(cuentaDePilar);
        CuentaPersonal cuentaDeBusqueda = null;
        //Buscamos la cuenta de Jose cuyo número de cuenta es 2
        cuentaDeBusqueda = registroDeCuentas.buscarCuenta(2);
        System.out.println("Cuenta de "+cuentaDeBusqueda.getNombreCliente()+" antes de
operar \n"+cuentaDeBusqueda);
        cuentaDeBusqueda.depositar(20.0);
        cuentaDeBusqueda.retirar(30.0); //+20-30
        System.out.println("Cuenta de "+ cuentaDeBusqueda.getNombreCliente()+" "
después de operar \n"+cuentaDeBusqueda);
        //Buscamos la cuenta de Pilar cuyo número de cuenta es 3
        cuentaDeBusqueda = registroDeCuentas.buscarCuenta(3);
        System.out.println("Cuenta de "+cuentaDeBusqueda.getNombreCliente()+" antes de
operar \n"+cuentaDeBusqueda);
        cuentaDeBusqueda.getNombreCliente();
        cuentaDeBusqueda.depositar(20.0);
        cuentaDeBusqueda.retirar(60.0); //30+20-60(-2.5)
        System.out.println("Cuenta de "+ cuentaDeBusqueda.getNombreCliente()+" después
de operar \n"+cuentaDeBusqueda);
    }
}
```

Resultado

```
<terminated> PruebaCuentas [Java Application] C:\Users\Victor Peiro\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_22.0.2.v20240802-1626\jre\bin\javaw.exe (14 nov 2024, 18:42:01 - 18:42)
Cuenta de Jose antes de operar
CuentaPersonal [numeroCuenta=2, nombreCliente=Jose, saldo=20.0][dia 31 de Diciembre de 2022, 2.0]
Cuenta de Jose después de operar
CuentaPersonal [numeroCuenta=2, nombreCliente=Jose, saldo=10.0][dia 31 de Diciembre de 2022, 2.0]
Cuenta de Pilar antes de operar
CuentaPersonal [numeroCuenta=3, nombreCliente=Pilar, saldo=30.0]CuentaCheque [numerosRojos=false, comisionChequera=3.0, comisionSaldoInsuficiente=0.05]
Cuenta de Pilar después de operar
CuentaPersonal [numeroCuenta=3, nombreCliente=Pilar, saldo=-69.5]CuentaCheque [numerosRojos=true, comisionChequera=3.0, comisionSaldoInsuficiente=0.05]
```